Helm & Kustomize

ONPK

2026

plectrum engineering

About me

- My name is Daniel Rajčan
- I have 10+ years of hands-on experience with cloud technologies
- I came across various technologies in companies from SMB to enterprises
- I went through technical and leadership tracks
- In 2023 with my friend we founded Plectrum Engineering
  - We love platform technologies and education



Before we begin

Lab pack

- git clone https://github.com/plectrum-engineering/fri-uniza-onpk-helmsman-starter.git

Tools

- A running cluster: `minikube start`
  - https://minikube.sigs.k8s.io/docs/start/?arch=%2Fmacos%2Farm64%2Fstable%2Fhomebrew
- `kubectl` (Kustomize is built in)
  - https://kubernetes.io/docs/tasks/tools/install-kubectl-macos/
- Helm 4
  - **https://helm.sh/docs/intro/install/**

- Pre-flight check:
  - `Mac/Linux: ./operation-helmsman-starter/00-setup/check-tools.sh`
  - `Windows: powershell -ExecutionPolicy Bypass -File ./operation-helmsman-starter/00-setup/check-tools.ps1`



Before we begin: output

```
➜ 00-setup git:(main) /bin/bash check-tools.sh
Pre-flight check
  [ OK ] kubectl v1.36.1
  [ OK ] helm v4.3.1+gc0d08d0
  [ OK ] Helm 4 detected
  [ OK ] kubectl kustomize (built-in)
  [ OK ] standalone kustomize v5.8.3 (optional)
  [ OK ] cluster reachable: minikube
All systems go. Welcome aboard, helmsman.
```



Agenda

01 Why packaging?
02 Helm fundamentals
03 Using charts
04 Writing charts
05 Release lifecycle
06 Kustomize fundamentals
07 Patches, generators & components
08 Helm vs Kustomize
09 Operation Helmsman - the missions



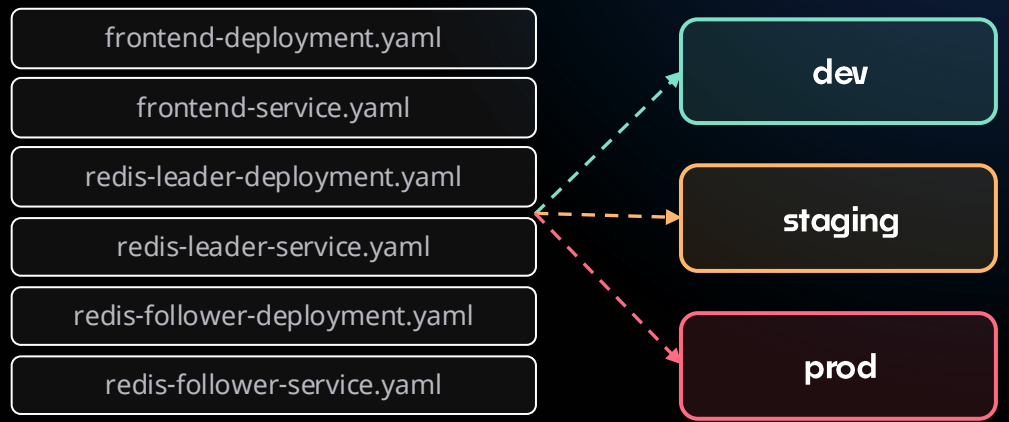
01

Why packaging?



Remember the guestbook?

- Last time you wrote **6 YAML files** by hand
- Now it is required to deploy it to **dev**, **staging** and **prod**
  - different replica counts, resources, images, URLs
- Copy the folder three times?
  - every fix has to be done 3x - and somebody always forgets one
  - no version, no history, no rollback, no "uninstall the whole app"



3 copies x 6 files = 18 files that drift apart



What we actually want

Parameters
one source, many environments

Versioning
"guestbook 1.4.2 runs in prod"

Lifecycle
install, upgrade, rollback, uninstall as one unit

Sharing
reuse what others built (databases, ingress, monitoring)



Two philosophies

Templating - Helm

- YAML with **placeholders** + a values file
- Logic: `if`, `range`, functions
- Packaged, versioned **charts**
- Release management built in

Overlays - Kustomize

- Plain, valid YAML **base**
- Environment **overlays** patch the base
- No templating language at all
- Built into `kubectl` (`-k`)

Not rivals - many teams use both. We will too.



02

Helm fundamentals



What is Helm?

- The **package manager for Kubernetes** - think `apt`, `brew` or `npm`, but for clusters
- CNCF graduated project, the de facto standard for distributing Kubernetes apps
- Helm can
  - find and install software others packaged
  - template your own manifests from a single values file
  - track every install and upgrade as a numbered **revision**
  - roll back to any previous revision with one command
- Pure client-side CLI - talks directly to the Kubernetes API
- Official web: https://helm.sh



Three big concepts

Chart
A package: templates + default values + metadata

Repository
A place where charts live: HTTP index or OCI registry

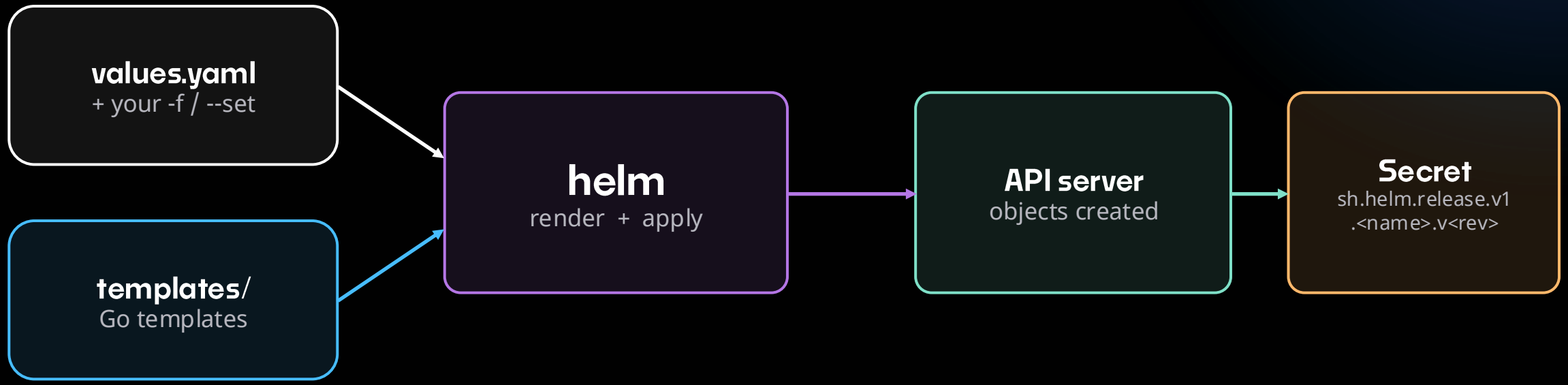
Release
One installed instance of a chart in a cluster

A **Chart** is pulled from a **Repository** and installed as a **Release**.
Same chart, installed twice = two releases (e.g. `guestbook-dev`, `guestbook-prod`).
Every change to a release creates a new **revision**: 1, 2, 3 ...



How Helm works

Helm renders plain manifests locally, sends them to the API server and stores the release record as a Secret in the release namespace.



Try it: `kubectl get secrets -l owner=helm -A`

03

Using charts



Where do charts come from?

Artifact Hub - https://artifacthub.io

- the search engine for charts (and Kustomize, OPA, Falco … packages)
- check: publisher, number of stars, last release, security report

Classic HTTP repositories

- an `index.yaml` + `.tgz` archives behind any web server
- `helm repo add podinfo https://stefanprodan.github.io/podinfo`

OCI registries (the modern way)

- charts stored like container images - GHCR, ECR, ACR, Harbor …
- `helm install demo oci://ghcr.io/stefanprodan/charts/podinfo`

Local charts (your own, or pulled to inspect)

- a chart directory or a packaged .tgz on disk, with no repo needed
- `helm create mychart -> helm install demo ./mychart`



Repository commands

```
# list local repositories cache
$ helm repo list

# register a repository and refresh the local index
$ helm repo add podinfo https://stefanprodan.github.io/podinfo
$ helm repo update
$ helm repo list

# search repositories you added / search Artifact Hub
$ helm search repo podinfo --versions
$ helm search hub redis

# look before you leap
$ helm show chart  podinfo/podinfo
$ helm show values podinfo/podinfo > defaults.yaml
$ helm pull podinfo/podinfo --untar       # read the templates locally
```



Installing a release

```
helm install paint podinfo/podinfo \
  -n mission1 --create-namespace \
  -f defaults.yaml \
  --set ui.color="#7b2ff7"

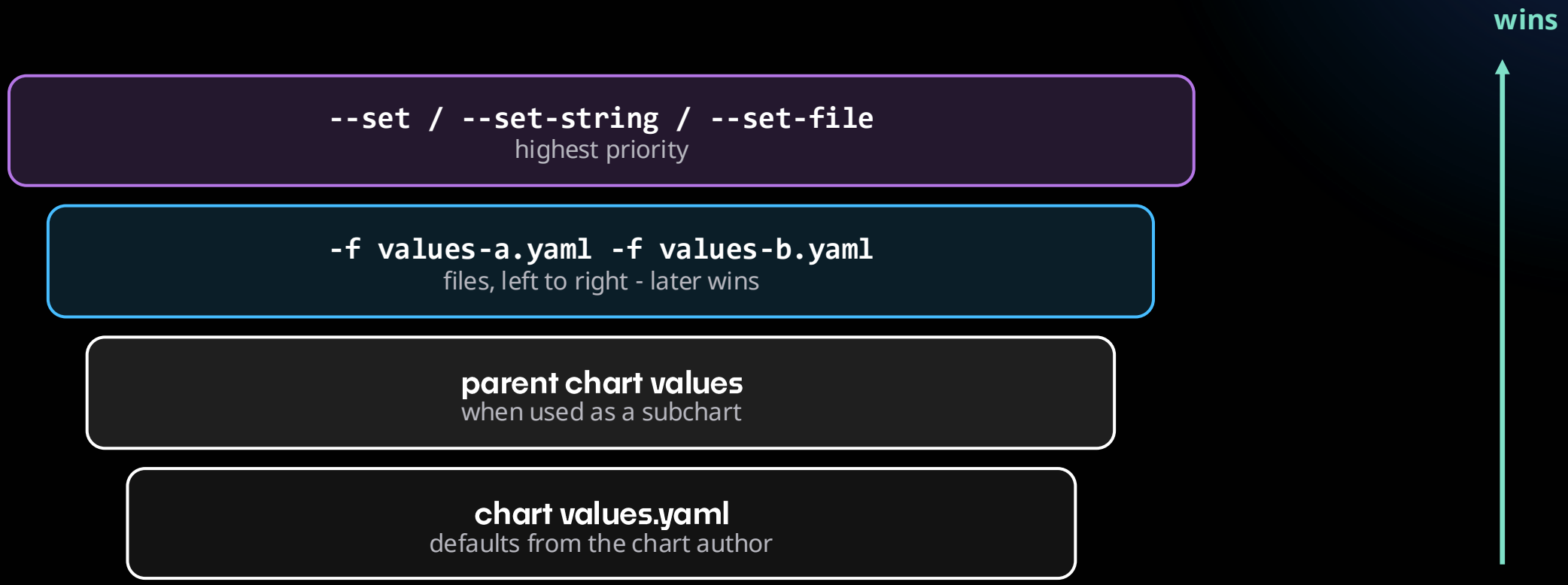
helm upgrade --install paint podinfo/podinfo \
  -n mission1 \
  --set ui.color="#34577c"

helm list –n mission1
```

- `-f` / `--values` - YAML file(s) with overrides, may be repeated
- `--set key=value` - quick overrides from the command line (nested keys with dots)
- `--version` - pin a chart version (**always** do this outside of the lab)



Values precedence



Inspecting releases

Command	Answers the question
`helm list -A`	What is installed where?
`helm status paint -n mission1`	Is it healthy? What did NOTES say?
`helm history paint –n mission1`	Which revisions exist and what happened?
`helm get values paint [--all] -n mission1`	Which values did we use?
`helm get manifest paint -n mission1`	What exactly was sent to the cluster?
`helm rollback paint 1 –n mission1`	Take me back to revision 1
`helm uninstall paint –n mission1`	Remove every object of the release



Mission 1 - Paint the pod

Folder 01-helm-consumer

- Add the podinfo repository and inspect its values
- Install release `paint` with **your name** and **your color** defined in `my-values.yaml`
- Open it: `kubectl -n mission1 port-forward svc/paint-podinfo 9898:9898`
- Upgrade: 3 replicas + a new colour via `--set` - refresh the browser
- Look at `helm history`, then **roll back** to revision 1
- Uninstall helm release



04

Writing charts



Chart anatomy

```
guestbook/
├── Chart.yaml            # name, version, appVersion
├── values.yaml           # default configuration
├── values.schema.json    # optional: validation
├── charts/               # dependencies (subcharts)
├── templates/
│   ├── _helpers.tpl      # named templates
│   ├── frontend-deployment.yaml
│   ├── frontend-service.yaml
│   ├── NOTES.txt         # printed after install
│   └── tests/
│       └── test-frontend.yaml
└── .helmignore
```

- Scaffold a new chart:
  - `helm create mychart`
- Files starting with `_` are not rendered as manifests
- Everything in `templates/` goes through the template engine
- Package it as one versioned `.tgz`



Chart.yaml

Chart.yaml

```yaml
apiVersion: v2              # chart API version
name: guestbook
description: PHP guestbook with Redis
type: application           # or: library
version: 0.1.0              # version of the CHART
appVersion: "v5"            # version of the APP
dependencies:               # optional subcharts
  - name: redis
    version: 21.x.x
    repository: oci://registry.example.com/charts
    condition: redis.enabled
```

- `version` follows **SemVer** - bump it on every chart change
- `appVersion` is informational - often used as default image tag
- `type: library` = only helpers, nothing to install



values.yaml

values.yaml

```yaml
environment: dev

frontend:
  replicas: 3
  image:
    repository: us-docker.pkg.dev/.../gb-frontend
    tag: v5
  service:
    type: ClusterIP
    port: 80
  resources:
    requests:
      cpu: 100m
      memory: 100Mi

ingress:
  enabled: false
  host: guestbook.local
```

- Values are the **public API** of your chart
- Group by component, use nested maps
- Sensible defaults - `helm install` with no flags should work
- Comment every key - it is the documentation
- Feature flags: `ingress.enabled: false`



Templates & built-in objects

Object	Contains	Example
`.Values`	merged values	`{{ .Values.frontend.replicas }}`
`.Release`	Name, Namespace, Revision, IsUpgrade	`{{ .Release.Name }}`
`.Chart`	everything from Chart.yaml	`{{ .Chart.AppVersion }}`
`.Capabilities`	cluster version and APIs	`{{ .Capabilities.KubeVersion }}`
`.Files`	non-template files in the chart	`{{ .Files.Get "conf/app.ini" }}`
`.Template`	current template name/path	`{{ .Template.Name }}`

Templates use the **Go template** language + 60+ functions from the **Sprig** library.

https://helm.sh/docs/chart_template_guide/builtin_objects



Template syntax essentials

```
# print a value
replicas: {{ .Values.frontend.replicas }}

# pipelines: value | function | function
image: {{ .Values.image.repository | quote }}
env: {{ .Values.environment | default "dev" | upper }}

# fail early with a clear message
host: {{ required "ingress.host is required" .Values.ingress.host }}

# whitespace control:  {{-  trims left,  -}}  trims right
labels:
  {{- include "guestbook.labels" . | nindent 4 }}

# local variables
{{- $port := .Values.frontend.service.port }}
```

Must-know functions

- `quote`, `default`, `required`
- `toYaml`, `nindent`, `indent`
- `include`, `tpl`, `printf`
- `b64enc`, `sha256sum`, `trunc`

Rule of thumb

- strings into YAML? `quote` them
- blocks into YAML? `toYaml | nindent N`

https://helm.sh/docs/chart_template_guide/function_list/



Flow control

if / else

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
...
{{- end }}

{{- if eq .Values.environment "prod" }}
replicas: 5
{{- else }}
replicas: 1
{{- end }}
```

with

```
# with: change scope, skip if empty
{{- with .Values.ingress.className }}
ingressClassName: {{ . }}
{{- end }}

# inside with/range, $ is the root
{{- with .Values.frontend }}
release: {{ $.Release.Name }}
replicas: {{ .replicas }}
{{- end }}
```

range

```
env:
{{- range $k, $v := .Values.env }}
  - name: {{ $k }}
    value: {{ $v | quote }}
{{- end }}

# values.yaml
env:
  LOG_LEVEL: debug
  FEATURE_X: "on"
```

`.` is the **current scope** - it changes inside `with` and `range`. `$` always points to the root.



Named templates - _helpers.tpl

templates/_helpers.tpl

```
{{/* Common labels added to every object */}}
{{- define "guestbook.labels" -}}
helm.sh/chart: {{ .Chart.Name }}-{{ .Chart.Version }}
app.kubernetes.io/part-of: guestbook
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}

{{/* repository@digest or repository:tag */}}
{{- define "guestbook.image" -}}
{{- if .digest -}}
{{ .repository }}@{{ .digest }}
{{- else -}}
{{ .repository }}:{{ .tag }}
{{- end -}}
{{- end }}
```

usage

```
metadata:
  labels:
    {{- include "guestbook.labels" . | nindent 4 }}
...
containers:
  - name: php-redis
    image: {{ include "guestbook.image" .Values.frontend.image }}
```

- `define` declares, `include` calls
- The 2nd argument is the **scope** you pass in
- Prefix names with the chart name - templates are global



From raw YAML to template

before - multi-tier challenge

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 3
  template:
    spec:
      containers:
        - name: php-redis
          image: us-docker.pkg.dev/.../gb-frontend:v5
          resources:
            requests:
              cpu: 100m
              memory: 100Mi
```

after - templates/frontend-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
  labels:
    {{- include "guestbook.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.frontend.replicas }}
  template:
    spec:
      containers:
        - name: php-redis
          image: {{ include "guestbook.image" .Values.frontend.image }}
          resources:
            {{- toYaml .Values.frontend.resources | nindent 12 }}
```



NOTES.txt

templates/NOTES.txt

```
Guestbook {{ .Release.Name }} runs in {{ .Release.Namespace }}
{{- if .Values.ingress.enabled }}
Open: http://{{ .Values.ingress.host }}
{{- else }}
Open: kubectl port-forward svc/frontend 8080:80
{{- end }}
```

NOTES are templated too - print **how to use** the release

helm test

templates/tests/test-frontend.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: "{{ .Release.Name }}-test-frontend"
  annotations:
    "helm.sh/hook": test
spec:
  restartPolicy: Never
  containers:
    - name: wget
      image: busybox:1.37
      command: ["sh", "-c"]
      args: ["wget -qO- http://frontend | grep -i guestbook"]
```

`helm test <release>` runs resource annotated ”helm.sh/hook”: test



Debugging toolbox

Command	Catches
`helm lint ./guestbook`	broken YAML, missing fields, bad Chart.yaml
`helm template gb ./guestbook`	renders locally - no cluster needed
`helm template ... --debug`	shows the broken output when YAML parsing fails
`helm install ... --dry-run=server`	server-side validation, nothing is created
`helm template ... | kubeconform -strict`	schema errors (wrong types, unknown fields)
`helm get manifest <release>`	what is really running

Golden loop: edit -> `helm template` -> `helm lint` -> `helm upgrade --install`



values.schema.json

values.schema.json

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["environment"],
  "properties": {
    "environment": {
      "type": "string",
      "enum": ["dev", "staging", "prod"]
    },
    "frontend": {
      "type": "object",
      "properties": {
        "replicas": { "type": "integer", "minimum": 1 }
      }
    }
  }
}
```

```
$ helm install gb ./guestbook --set frontend.replicas=0
Error: values don't meet the specifications:
- at '/frontend/replicas': minimum: got 0, want 1
```

- Validated on `install`, `upgrade`, `lint`, `template`
- Typos and wrong types fail **before** they reach the cluster
- Free documentation for IDEs



Classic pitfalls

Indentation

- `indent` vs `nindent` - and the exact number of spaces

Types

- env values must be strings: `value: {{ .Values.port | quote }}`

Missing keys

- `.Values.servce.port` -> nil pointer error; one typo, whole render fails

Immutable fields

- changing `spec.selector` labels breaks `helm upgrade` - keep selectors stable

Names

- max 63 characters - use `trunc 63 | trimSuffix "-"`



Mission 2 - Chart the guestbook

Folder 02-guestbook-chart (starter chart provided)

- `frontend-deployment.yaml` is already templated - use it as the example
- M2.1 Template the remaining 5 manifests from `values.yaml`
- M2.2 Write the `guestbook.image` helper
- M2.3 Add an `onpk.io/environment` label to the common labels
- M2.4 Make NOTES.txt useful (port-forward vs NodePort vs Ingress)
- M2.5 Optional Ingress behind `ingress.enabled`
- M2.6 A `helm test` that greps the guestbook page
- M2.7 `values.schema.json` that rejects `replicas: 0`
- M2.8 Prove your schema works: try `frontend.replicas=0`

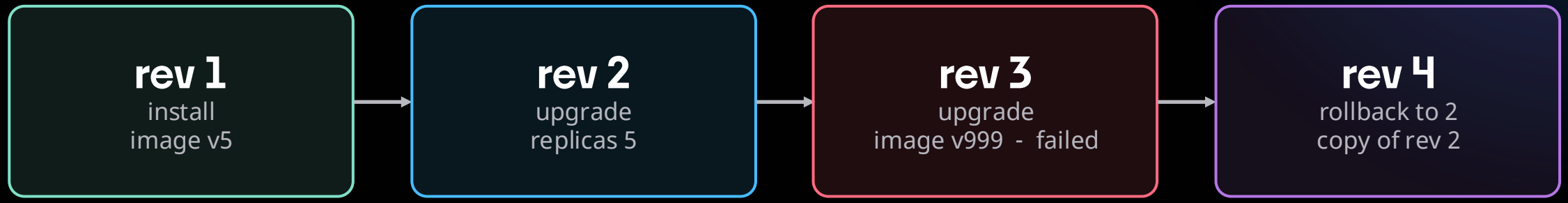


05

Release lifecycle



Revisions




```
$ helm history gb -n guestbook-prod
REVISION  STATUS      CHART            DESCRIPTION
2         superseded  guestbook-0.1.0  Upgrade complete
3         failed      guestbook-0.1.0  Upgrade "gb" failed: context deadline exceeded
4         deployed    guestbook-0.1.0  Rollback to 2
```



Upgrade like a pro

```
$ helm upgrade --install gb ./guestbook -n guestbook-prod -f values-prod.yaml --rollback-on-failure --timeout 90s
```

Flag	Meaning
`--install`	install if the release does not exist yet
`--wait / --wait=watcher`	wait until resources are ready (kstatus)
`--rollback-on-failure`	roll back automatically if the upgrade fails (was --atomic)
`--reset-then-reuse-values`	keep previous values, but take new chart defaults



Hooks

Hook	Runs
`pre-install / post-install`	before / after objects are created
`pre-upgrade / post-upgrade`	around every upgrade
`pre-delete / post-delete`	around uninstall
`pre-rollback / post-rollback`	around rollback
`test`	only on helm test

DB migration Job

```yaml
metadata:
  annotations:
    "helm.sh/hook": pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy":
        before-hook-creation,hook-succeeded
```

Typical use: database migrations, backups before upgrade, smoke tests.



Dependencies - subcharts

Chart.yaml

```yaml
# Chart.yaml
dependencies:
  - name: postgresql
    version: "16.x.x"
    repository: oci://registry.example.com/charts
    condition: postgresql.enabled
```

values.yaml

```yaml
# values.yaml - configure the subchart under its name
postgresql:
  enabled: true
  auth:
    database: guestbook
```

- Download them into `charts/`:
  - `helm dependency update`
- `Chart.lock` pins exact versions - commit it
- Parent values override subchart defaults
- `condition` turns a dependency on/off



Packaging & distribution

```
$ helm package ./guestbook                          # -> guestbook-0.1.0.tgz
$ helm registry login ghcr.io -u <user>
$ helm push guestbook-0.1.0.tgz oci://ghcr.io/<org>/charts
$ helm install gb oci://ghcr.io/<org>/charts/guestbook --version 0.1.0
```

- A chart version is **immutable** - change anything, bump `version`



Mission 3 - Release day

Folder 03-release-day

- M3.1 Write `values-dev.yaml` and `values-prod.yaml`, install both into separate namespaces
- M3.2 **The bad release**: upgrade prod to image tag `v999` with `--rollback-on-failure --timeout 90s`
  - watch the pods - what does Helm do when the timeout hits?
- M3.3 `helm package` the chart and push it to ghcr.io



06

Kustomize fundamentals

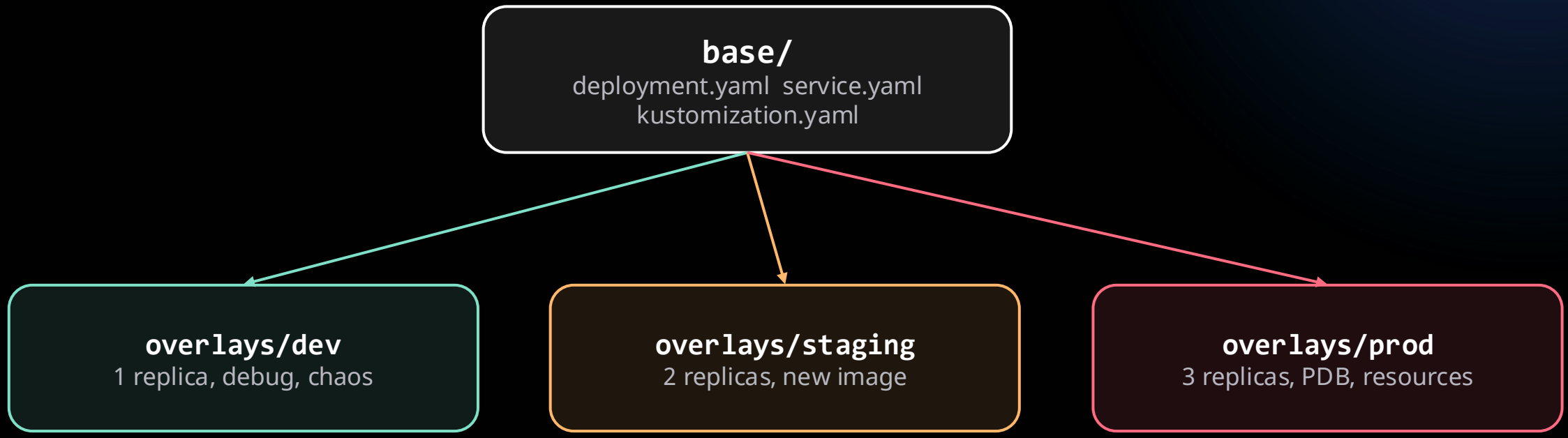


What is Kustomize?

- **Template-free** customization of Kubernetes YAML
- Your files stay **valid Kubernetes manifests** - no placeholders, no new language
- You declare **what to change**, Kustomize computes the result
- Built into kubectl since 1.14
  - `kubectl kustomize ./overlays/dev  # render`
  - `kubectl apply -k ./overlays/dev  # render + apply`
- Standalone binary `kustomize` (v5.8) is newer and has extra features
- Natively supported by Argo CD and Flux



Base & overlays



An overlay **references** the base and adds its own changes. The base never knows about the overlays.



kustomization.yaml

overlays/prod/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:            # files, dirs, other kustomizations, URLs
  - ../../base
  - pdb.yaml

namespace: podinfo-prod
namePrefix: prod-
labels:
  - pairs:
      env: prod

replicas:
  - name: podinfo
    count: 3

images:
  - name: ghcr.io/stefanprodan/podinfo
    newTag: 6.15.0
```

- One `kustomization.yaml` per directory
- `resources` can point to another kustomization - that is how layering works
- Remote bases are possible:
  - `github.com/org/repo//kustomize?ref=v1.2`



Built-in transformers

Field	Does	Watch out
`namespace`	sets metadata.namespace everywhere	also updates RoleBinding subjects
`namePrefix / nameSuffix`	renames objects + all references	Services used by DNS get renamed too
`labels`	adds labels (optionally to selectors)	includeSelectors: true = immutable!
`commonAnnotations`	adds annotations everywhere	-
`images`	changes name / tag / digest	matches by image name
`replicas`	sets replica count	no patch needed

https://kubectl.docs.kubernetes.io/references/kustomize/kustomization/



Render, diff, apply

```
# see the result - always do this first
$ kubectl kustomize overlays/prod

# what would change in the cluster?
$ kubectl diff -k overlays/prod

# apply / delete everything of an overlay
$ kubectl apply -k overlays/prod
$ kubectl delete -k overlays/prod

# standalone binary, plus Helm chart support
$ kustomize build --enable-helm overlays/prod
```



07

Patches, generators & components



Strategic merge patch

overlays/prod/resources-patch.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo          # which object
spec:
  template:
    spec:
      containers:
        - name: podinfod  # merge key
          resources:
            requests:
              cpu: 200m
            limits:
              memory: 256Mi
```

kustomization.yaml

```yaml
patches:
  - path: resources-patch.yaml
```

- Looks like a partial Kubernetes object
- Lists are merged by key (`name` for containers)
- Best for adding or changing fields



JSON 6902 patch

kustomization.yaml

```yaml
patches:
  - target:
      kind: Deployment
      name: podinfo
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/command/3
        value: --level=warn
      - op: remove
        path: /spec/template/spec/containers/0/ports/1
      - op: add
        path: /metadata/annotations/team
        value: blue
```

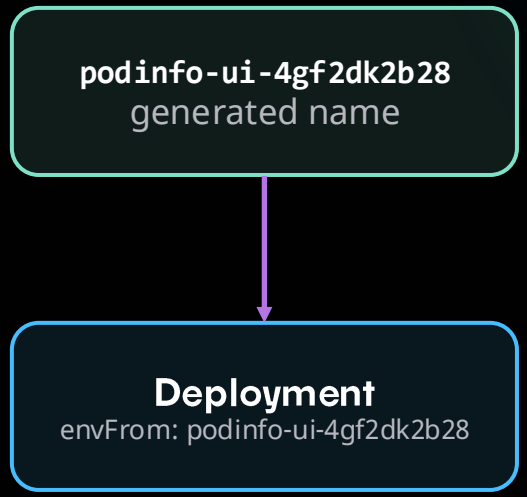
- Precise operations: `add`, `remove`, `replace`, `move`, `copy`, `test`
- Address list items **by index**
- `target` can select many objects: by kind, name, labels
- Use it when strategic merge cannot express the change



Generators & the hash suffix

kustomization.yaml

```yaml
configMapGenerator:
  - name: podinfo-ui
    literals:
      - PODINFO_UI_MESSAGE=Hello from DEV
      - PODINFO_UI_COLOR=#2e7d32
secretGenerator:
  - name: api
    files: [token.txt]
```



- content changes
- -> new hash
- -> new name
- -> pods roll

- References are rewritten automatically - a config change **always** triggers a rollout
- Overlays can extend a generator from the base with `behavior: merge`

Components - reusable features

component

```yaml
# components/chaos-monkey/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1alpha1
kind: Component
patches:
  - target: { kind: Deployment, name: podinfo }
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/command/5
        value: --random-error=true
```

opt-in

```yaml
# overlays/dev/kustomization.yaml
resources:
  - ../../base
components:
  - ../../components/chaos-monkey
```

- A **mix-in**: switch features on per overlay
- Examples: monitoring, TLS, maintenance mode, debug sidecar
- Avoids the diamond problem of overlays-of-overlays



Replacements

kustomization.yaml

```yaml
replacements:
  - source:
      kind: Service
      name: backend
      fieldPath: metadata.name          # e.g. golf-backend (after namePrefix)
    targets:
      - select:
          kind: Deployment
          name: web
        fieldPaths:
          - spec.template.spec.containers.[name=nginx].env.[name=BACKEND_HOST].value
```

- Copy a value from one object into another - runs **after** prefixes, so you get the final name
- Replaces the deprecated `vars`



Mission 4 - Three environments

Folder 04-kustomize (base is done, overlays are yours)

- M4.1 **dev** - namespace, `dev-` prefix, label, merged UI message and colour
- M4.2 **staging** - 2 replicas and image 6.15.0 without writing a single patch
- M4.3 **prod** - strategic merge patch for resources, JSON patch for the log level, PDB
- M4.4 **chaos-monkey** component - podinfo starts failing randomly; enable it only in dev
- M4.5 **maintenance** component - flip prod into maintenance mode and watch the rollout
- Check every overlay with `kubectl kustomize` before `kubectl apply -k`



08

Helm vs Kustomize



Side by side

	Helm	Kustomize
Approach	templating + values	overlays + patches
Input	chart (not valid YAML)	plain valid YAML
Logic	if / range / functions	none - declarative only
Packaging	versioned charts, repos, OCI	directories, Git refs
Lifecycle	releases, history, rollback, hooks	none - kubectl apply
Learning curve	Go templates + Sprig	a few YAML fields
Sweet spot	distributing apps to others	your own manifests across envs



Better together




```yaml
helmCharts:
  - name: podinfo
    repo: https://stefanprodan.github.io/podinfo
    version: 6.15.0
    releaseName: storefront
    valuesInline: { replicaCount: 2 }
```

- "Last mile" changes the chart author did not foresee
- Needs `--enable-helm`
- Alternative: Helm post-renderer plugin



Which one when?

Reach for Helm when

- you ship software to **other teams or customers**
- you need install / upgrade / rollback as a unit, or hooks
- you consume third-party software - it is almost always a chart

Reach for Kustomize when

- it is **your** app and you only need per-environment differences
- you want reviewers to read plain YAML in pull requests

Use both when

- you install vendor charts but need small, local tweaks



Key takeaways

- Both tools only **produce YAML** - Kubernetes never sees Helm or Kustomize
- **Helm** = templates + values + release lifecycle; values are your chart's API
- **Kustomize** = plain base + overlays; patches, generators and components
- Always render before you apply: `helm template`, `kubectl kustomize`
- Pin versions: chart `--version`, image tags or digests, `?ref=` for remote bases
- Helm 4 is current - `--rollback-on-failure` replaced `--atomic`



Side quests - pick your adventure

Boss level
05-boss-franchise

White-label podinfo for two clients:
Helm chart + Kustomize overlays

Escape room
06-escape-room

The lighthouse chart has 8 locks.
Fix them to reveal the code word

Kustomize golf
07-kustomize-golf

Match expected.yaml exactly -
in as few lines as possible

Each quest has a self-check script.



plectrum
engineering

Thank You!